

Reviewer Pack — Minimal Rank of Twisted Group Algebras vs Monotone Circuit D...

Entry 65492e207bca · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 01:23:08 UTC

Minimal Rank of Twisted Group Algebras vs Monotone Circuit Depth for k-CLIQUE

Entry ID: 65492e207bca **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-26 01:23:08 UTC

1. Conjecture

Field A (mathematical branch): Twisted Group Algebra Theory **Field B** (complexity object): Complexity Theory: Monotone Circuit Complexity

Statement:

[‘For a given n , let G be a finite group and consider its twisted group algebra A_G over a field F with characteristic p . The minimal rank of the twisted group algebra for a k -CLIQUE instance on n variables is bounded below by $2^{(n-k) \cdot k}$.’, ‘Equivalently, there exists an absolute constant $c > 0$ such that for all twisted group algebras A_G and k -CLIQUE instances with n variables, if the minimal rank of A_G is less than $2^{(n-k) \cdot k}$, then the monotone circuit depth required to compute k -CLIQUE exceeds cn .’]

Rationale (proposer’s reasoning):

[‘Twisted group algebras provide a rich algebraic structure that can encode combinatorial properties of graphs. By associating twisted group algebras with k -CLIQUE instances, we aim to find a new invariant that can capture the complexity of k -CLIQUE.’, ‘If this conjecture holds, it would suggest that certain algebraic properties of twisted group algebras are closely related to the computational complexity of k -CLIQUE, potentially leading to new algorithmic approaches.’]

Taxonomy category: MONOTONE_CLIQUE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): cedee71d6ba47a2e

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all $n \leq 40$, the mean of the ratio between the computed minimal rank and $2^{(n-k) \cdot k}$ across 30 random seeds is greater than or equal to 0.5 and no seed produces a ratio less than 0.3.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "twisted group algebra" AND minimal rank" - "monotone circuit complexity" AND "k-CLIQUE" AND minimal rank" - "algebraic complexity theory" AND twisted group algebra AND monotone circuits"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_k_clique_instance(n, k):
    if n < k:
        return None
    vertices = list(range(n))
    edges = set()
    for i in range(k):
        for j in range(i + 1, k):
            edges.add((vertices[i], vertices[j]))
    for _ in range(math.comb(n - k, 2)):
        u, v = random.sample(vertices[k:], 2)
        if (u, v) not in edges and (v, u) not in edges:
            edges.add((u, v))
    return vertices, edges

def twisted_group_algebra(instance):
    n, edges = instance
    G = {i: i for i in range(n)}
    algebra = [[0] * n for _ in range(n)]
    for u, v in edges:
        algebra[u][v] = algebra[v][u] = -1
    return algebra

def minimal_rank(algebra):
    n = len(algebra)
    rank = 0
    for i in range(n):
```

```

        if all(algebra[i][j] == 0 for j in range(i)):
            continue
        pivot_row = i
        for j in range(i + 1, n):
            if algebra[j][i] != 0:
                pivot_row = j
                break
        if algebra[pivot_row][i] == 0:
            continue
        rank += 1
        for j in range(n):
            algebra[i][j], algebra[pivot_row][j] = algebra[pivot_row][j], algebra[i][j]
        for j in range(n):
            if i != j and algebra[j][i] != 0:
                factor = Fraction(algebra[j][i], algebra[i][i])
                for k in range(n):
                    algebra[j][k] -= factor * algebra[i][k]

    return rank

def monotone_circuit_depth(k, n):
    # Simplified heuristic based on known lower bounds
    return 2 ** (n - k) + k

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.choice([5, 10, 15, 20, 30, 40])
    k = random.randint(2, min(n // 2, 5))
    instance = generate_k_clique_instance(n, k)
    if instance is None:
        return {
            "metric_name": "minimal_rank",
            "metric_value": 0,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "k > n"
        }
    algebra = twisted_group_algebra(instance)
    rank = minimal_rank(algebra)
    depth = monotone_circuit_depth(k, n)
    ratio = Fraction(rank, 2 ** (n - k) * k)
    return {
        "metric_name": "minimal_rank",
        "metric_value": float(ratio),
        "instances_tested": 1,
        "conjecture_holds": ratio >= 0.5 and ratio < 0.3,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2**i + 1 for i in range(5, 8)]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_
        results.append(result)

```

```

mean_ratio = sum(r["metric_value"] for r in results) / len(results)
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)
if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_ratio} std=0.0 support_fraction={support_fraction}")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_ratio} std=0.0 support_fraction={support_fraction}")
else:
    first_failing_seed = next(i for i, r in enumerate(results) if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={seeds[first_failing_seed]}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c7262ab4.py", line 97, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c7262ab4.py", line 80, in run_trial
    algebra = twisted_group_algebra(instance)
              ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c7262ab4.py", line 34, in twisted_group
    G = {i: i for i in range(n)}
        ^^^^^^^
TypeError: 'list' object cannot be interpreted as an integer

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it did not complete its execution to verify the conjecture. | next: Re-run the test with proper error handling and ensure that it completes without crashing.

11. Audit log (LLM calls)

Total LLM calls: 12

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13209
2	propose	ollama_remote	glm4:latest	0	0	10482
3	propose	ollama_remote	glm4:latest	0	0	11289

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
4	propose	ollama_remote	glm4:latest	0	0	11403
5	preregistration	ollama_remote	glm4:latest	0	0	5734
6	novelty	ollama_remote	glm4:latest	0	0	4645
7	novelty	ollama_remote	glm4:latest	0	0	5520
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14415
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9174
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11528
11	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11964
12	judge	ollama_remote	glm4:latest	0	0	7931

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 117293 ms total latency. Provider mix: {'ollama_remote': 12}

(full prompt+response transcripts available in `research/audit/65492e207bca.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/65492e207bca.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/65492e207bca.tar.gz` (if generated)